Data-Structures & Algorithms

with

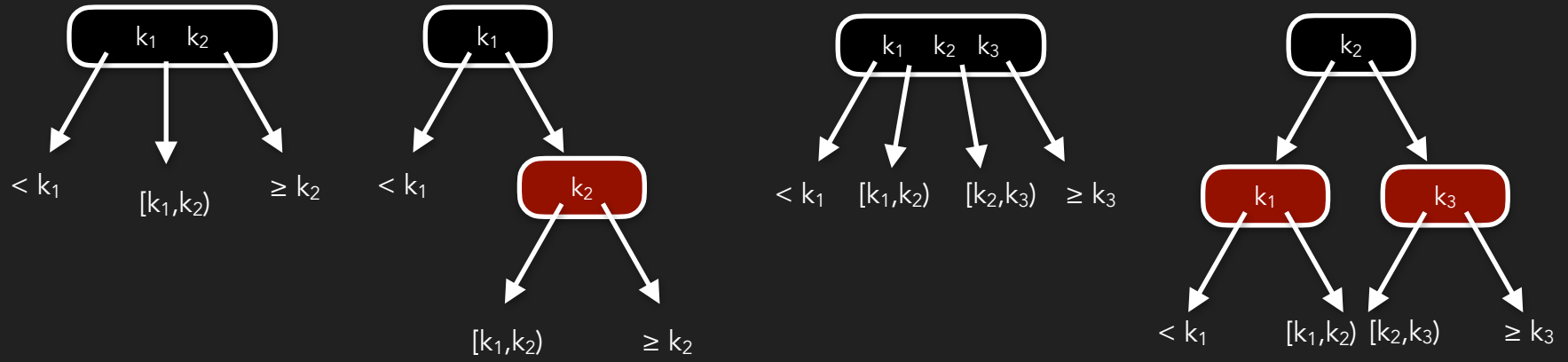
Manoj Prabhakaran

Lecture 30

Balanced BST (ctd.)

Recall

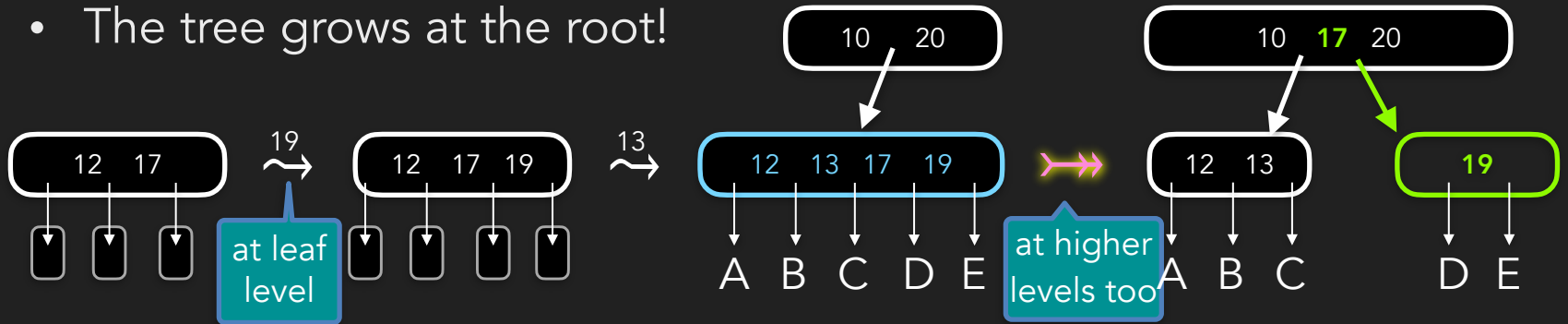
- A Red-Black tree is a BST with every node coloured black or red, and:
  - Every red node has a black parent
  - Every node has a unique *black-height*
- A 2-3-4 tree is a perfectly balanced search tree, with d-ary nodes, d∈{2,3,4}
- A 2-3-4 tree can be "translated" into a red-black tree



- How do you keep a 2-3-4 tree perfectly balanced?

Keeping 2-3-4 Trees Balanced: Insertion

- ASearch for the smallest interval where the key would go (at the bottom-most level), and insert the key into that node, increasing the node's number of children (2-ary → 3-ary, or 3-ary → 4-ary)
  - If it was already 4-ary (had 3 keys), split into two and push one of its keys up to its parent (recursively).
  - If root splits, form a new root
    - The tree grows at the root!



Attendance Break!

Login using CSE LDAP id at

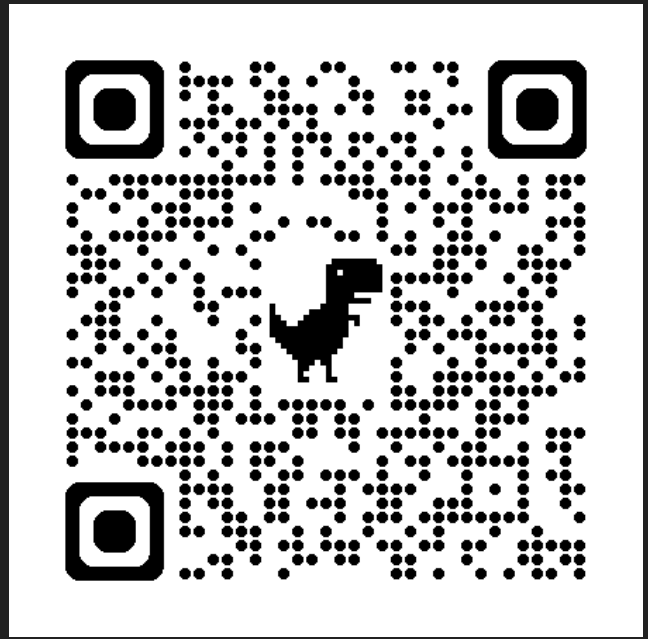
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!

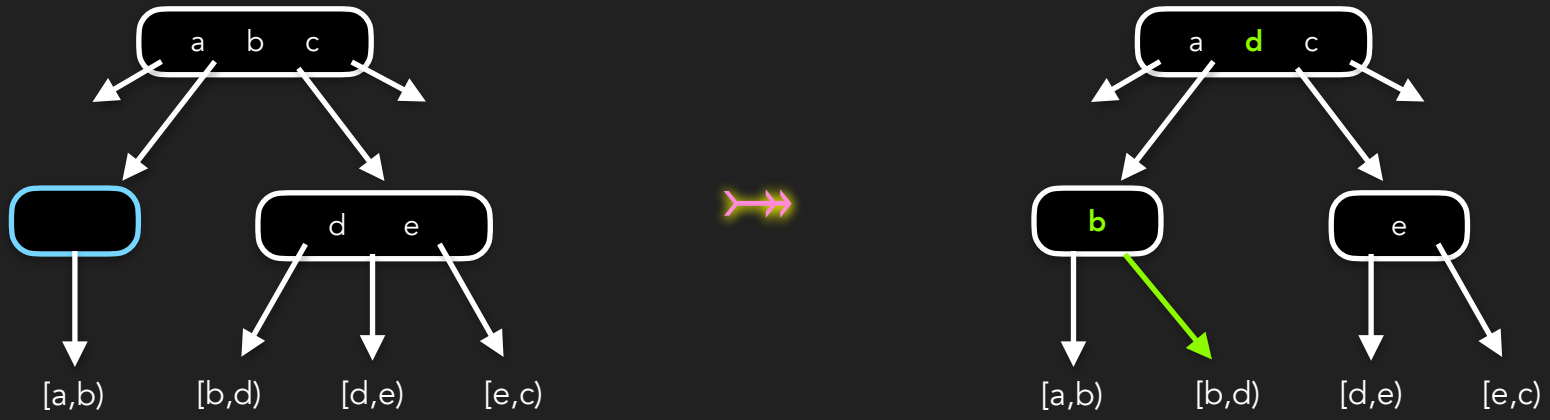


Keeping 2-3-4 Trees Balanced: Deletion

- To delete a key not at the bottom-most level, replace it with its in-order successor and delete that key (which is at the bottom-most level)
- To delete a key from the bottom-most level, first remove it from its node
  - If that node has no keys left (only one child), fix it by either:
    - *Transferring* a child from a 3-ary or 4-ary adjacent sibling; or,
    - *Merging* with a 2-ary adjacent sibling, by pulling the separating key down from the parent, to form a 3-ary node
      - If the parent becomes empty, recursively repopulate it, unless it is the root, in which case make this node the root.

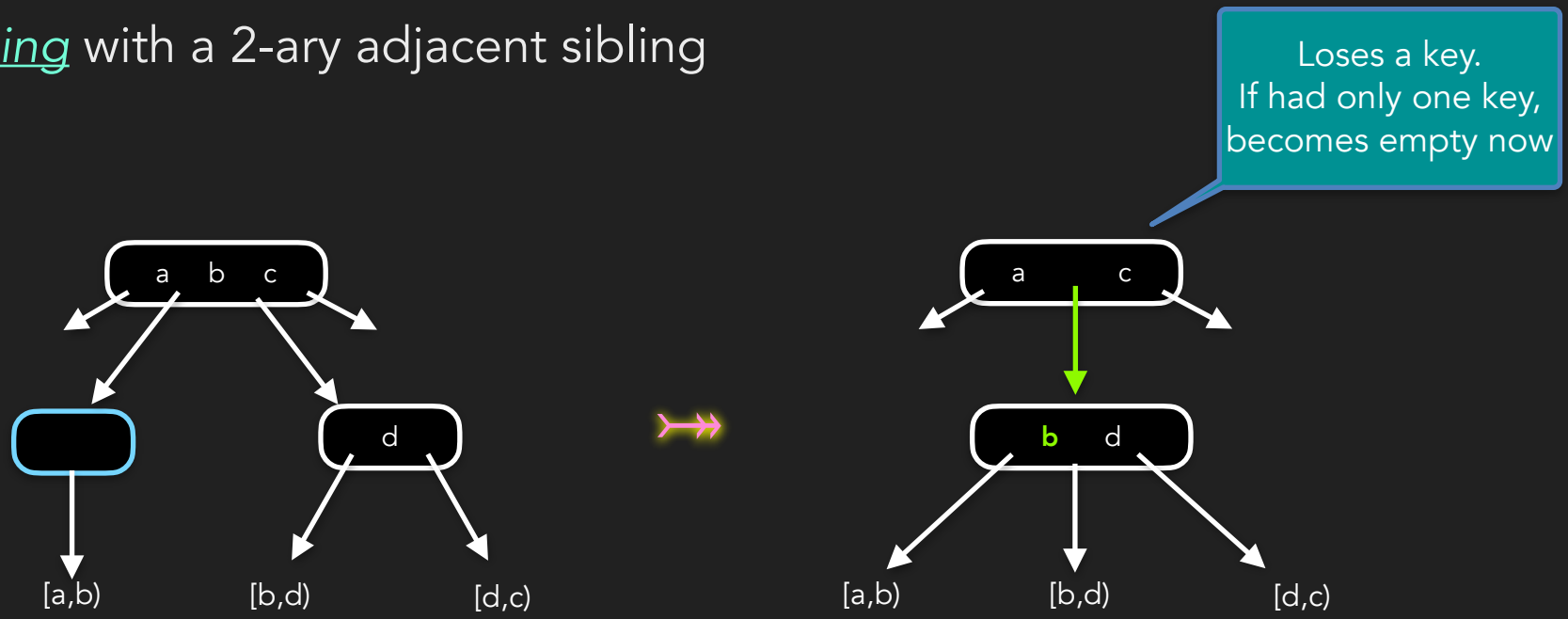
Keeping 2-3-4 Trees Balanced: Deletion

- Eliminating an empty node (with no keys and a single child)
- *Transferring* a child from a 3-ary or 4-ary adjacent sibling



Keeping 2-3-4 Trees Balanced: Deletion

- Eliminating an empty node (with no keys and a single child)
- *Merging* with a 2-ary adjacent sibling

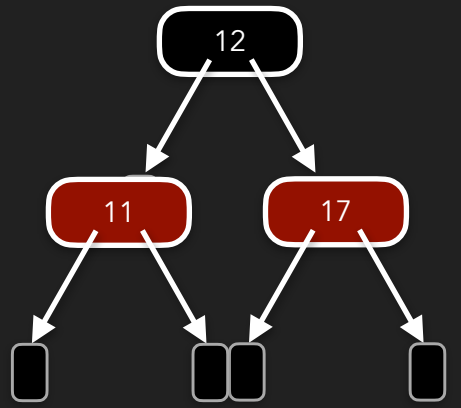


Translating to Red-Black Tree Operations

Insertion

- All the operations above can be translated to the binary red-black tree representation
- We will see a few examples (many similar cases omitted)
- Recall insertion: Find the bottom-most interval containing the new key and insert into that interval
  - In the binary tree, inserting into an interval translates to creating a child node
  - So insert normally, and colour it red

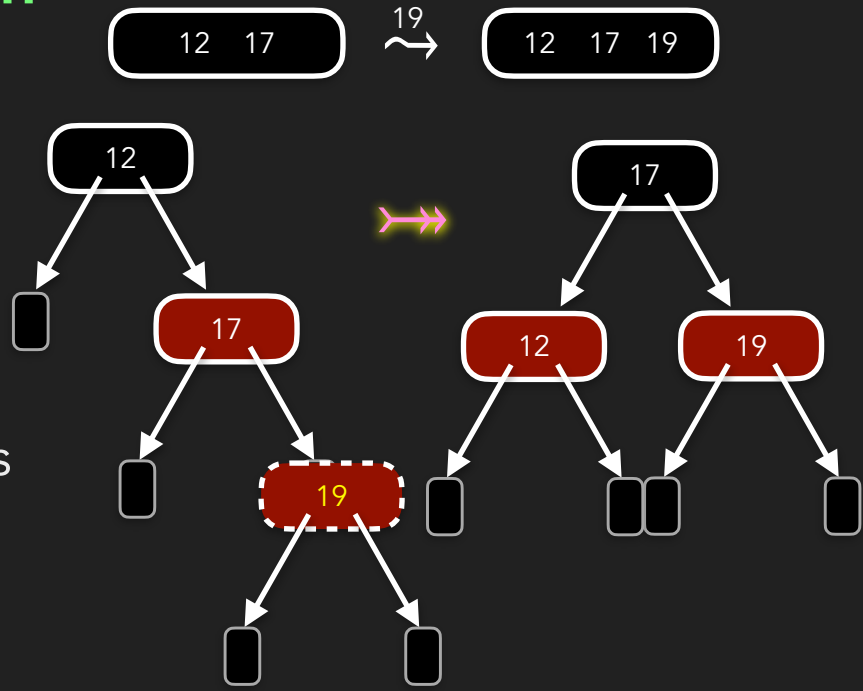




Translating to Red-Black Tree Operations

Insertion

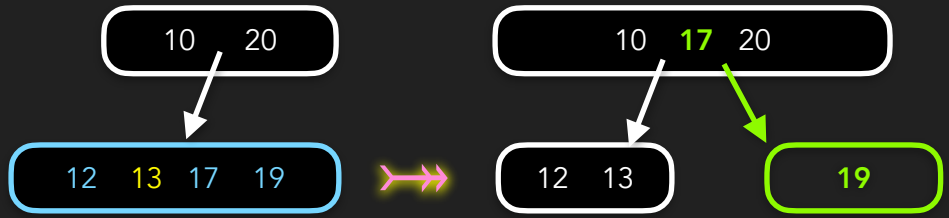
- But we *may* need a bit of adjusting
- If parent is red and parent-sibling is black, we have inserted into a 3-ary node expanded the "wrong way"
  - Switching the expansion corresponds to a "rotation" in the binary tree
- Not an issue when we are inserting into a 2-ary node (in 2-3-4 tree it goes from 2-ary to 3-ary; in red-black tree, a red child added to a black node)



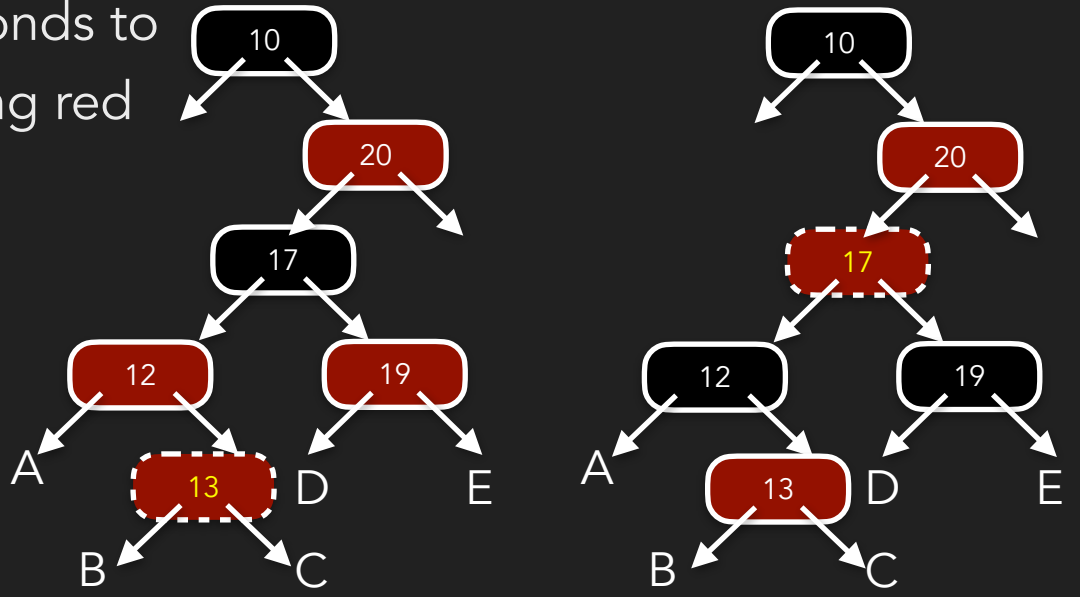
Translating to Red-Black Tree Operations

Insertion

- If inserting into a 4-ary node, we need to split (and recursively insert a key into the parent node)



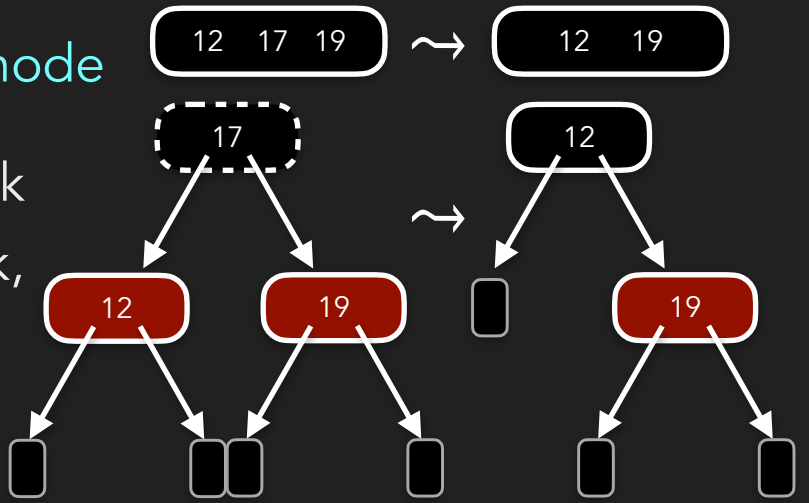
- In red-black tree, this corresponds to parent and parent-sibling being red
  - Splitting is just recolouring



Translating to Red-Black Tree Operations

Deletion

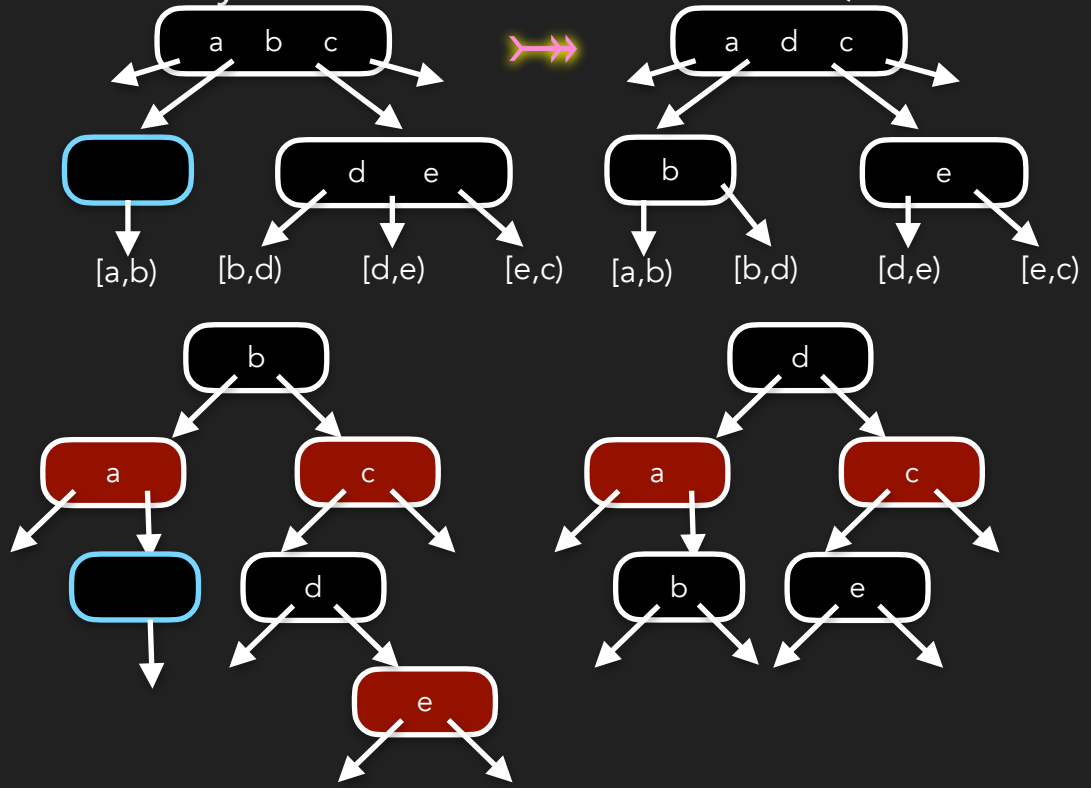
- Recall main step of deletion in 2-3-4 tree: delete from a bottom-most node
- When deleting a key from a 3-ary or 4-ary node
  - In red-black tree, if deleted node is black promote one of its red children as black, else simply remove it
- When deleting a key from a binary node
  - Need to repair an empty black node
  - Done recursively (as it can make the parent in 2-3-4 tree empty)



Translating to Red-Black Tree Operations

Deletion

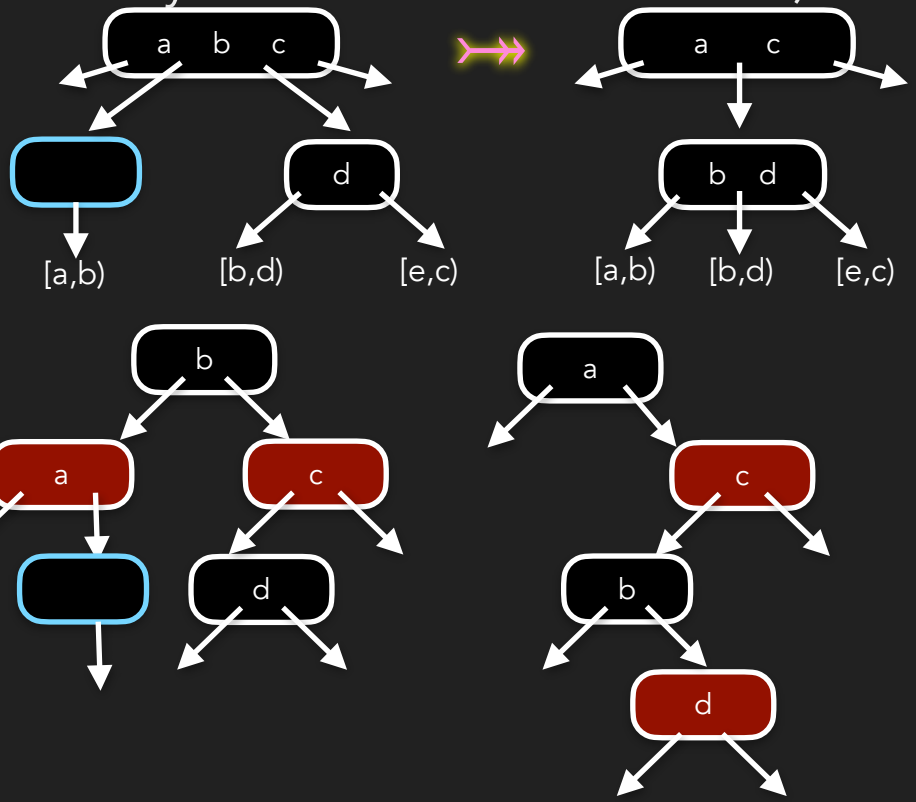
- Repairing an empty node (not necessarily at the bottom-most level)
  - If an adjacent sibling has ≥3 children, transfer a child
  - In red-black tree, a rotation involving an ancestor (possibly a grandparent)



Translating to Red-Black Tree Operations

Deletion

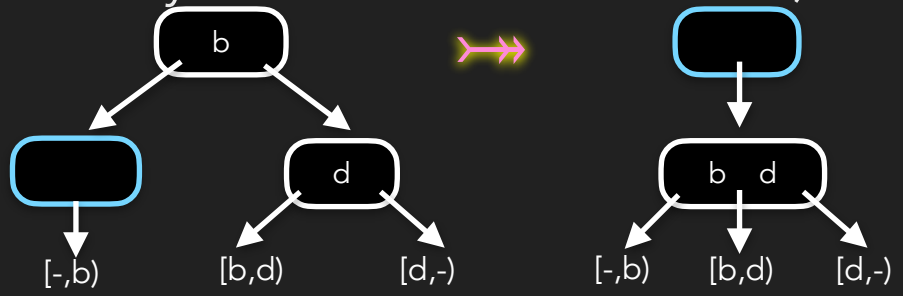
- Repairing an empty node (not necessarily at the bottom-most level)
  - If no adjacent sibling with ≥3 children, then there must be an adjacent sibling with 2 children: merge with it



Translating to Red-Black Tree Operations

Deletion

- Repairing an empty node (not necessarily at the bottom-most level)
  - If no adjacent sibling with ≥3 children, then there must be an adjacent sibling with 2 children: merge with it



  - Merger can cause the parent in the 2-3-4 tree (a black ancestor in the red-black tree) to be empty
    - If it is the root, simply remove it
    - Else, recursively repair it

